

商品搜索：从业务漏斗倒推技术选型

用 LIKE / ES / ES + Milvus 三档能力，撑住 gomall 最宽的一跳入口

gomall · deck 03 (v2 expanded)

业务定位：搜索是漏斗最宽一跳

ES 升级：从词重合到相关性，给商家与运营留旋钮

5min 上架可搜 SLA：商家最关心的事

ES 挂了不能 5xx：降级与运营 SOP

Hybrid 检索：把”商家不会写 SEO”的锅自己背了

客服话术：三个真实客诉怎么答

业务边界、容量与个性化路线图

业务侧总结：四角色 KPI 全打通

业务定位：搜索是漏斗最宽一跳

先回答一个业务问题：为什么搜索值得单独立项

- 用户进站后的**第一动作**：60% 直接点搜索框——比类目、推荐、首页 Banner 都靠前。
- C 端体感最直接：搜不到 = 没卖；搜偏 = 货架乱；搜慢 = 应用卡。三种坏体验都是**秒级流失**。
- 商家 KPI 锚点：商家上架的下一秒，最关心的是”我的货搜得到吗”。
- 运营 GMV 杠杆：召回 / 排序 / 加权三个旋钮，任一动 1% CTR，对应今日 GMV 涨跌都是六位数。

搜索系统的对外契约同时绑定 4 个角色 KPI，不是普通”功能页”，而是**业务入口**。

routes/router.go:41 /product/search 关键词入口；:42 /product/semantic-search hybrid 入口

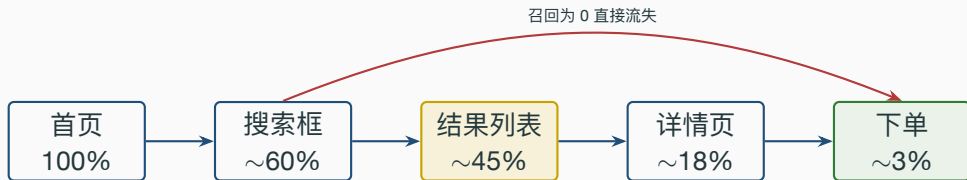
五角色对”搜索”四件事的关心

角色	关心的事	在 gomall 的对应能力
C 端用户	搜得到 / 搜得准 / 搜得快	ES + Milvus hybrid / p95 < 200ms
商家	”我的货 5min 内搜得到吗”	outbox + indexer 5min SLA
运营	”今日特卖能不能强插第一屏”	function_score / 字段加权 / boost
客服	”用户说搜不到，给个交代”	业务码 + 兜底 LIKE + 重建命令
SRE	”ES 挂了主站会不会一起挂”	静默降级 + 三档降级树

- 4 件事并不矛盾：用户要”准”、商家要”快”、运营要”调”、客服要”答”、SRE 要”稳”。
- 后面 35 页都是围绕这五个角色拆开讲。

业务码表 `pkg/e/code.go:46-58` (50001 / 60002 / 70001 / 70002)

用户漏斗：每一跳的业务意义



- **首页 → 搜索框 (100% → 60%)**: 用户主动表达意图，是**高价值流量**。
- **搜索框 → 列表 (60% → 45%)**: 列表 0 命中 / 空页 → 这一跳直接断。
- **列表 → 详情 (45% → 18%)**: 排序差 / 缩略图差 → 用户翻三页就退出。
- **详情 → 下单 (18% → 3%)**: 搜索质量决定”用户看到的是不是想要的”。

stressTest/REPORT.md: /product/show 同库 PK 查询 62,226 RPS / p95 3ms 为时延锚点

搜不到 = GMV 静默漏走（定量算账）

假设：日 UV 100 万 / 客单 200 元 / 当前漏斗系数 = $60\% \times 75\% \times 40\% \times 17\% =$

	召回率变化	漏斗系数	日 GMV 影响
	基线 100%	3.06%	612 万
3.06%	召回 -1pp $\rightarrow$ 99%	3.03%	-6 万 / 日
	召回 -5pp $\rightarrow$ 95%	2.91%	-30 万 / 日
	召回 -10pp $\rightarrow$ 90%	2.76%	-60 万 / 日

- 召回率每损 1pp，下游所有跳都按比例缩水——**杠杆效应明显**。
- ES 不上中文分词、Milvus 不接长尾语义，召回率直接掉到 60%–80% 区间。
- 这就是为什么 hybrid 检索值得花成本：不是炫技，是**止损**。

service/search/semantic.go:1-56 hybrid 主入口与默认权重

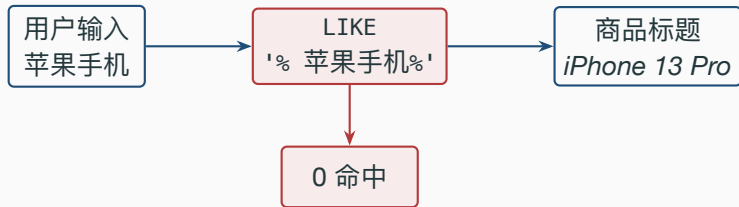
业务对搜索的四条硬约定

- 召回：用户搜得到的词，能跑出至少一条相关结果（C 端）。
- 时延：p95 < 200ms，列表页才不会“卡半秒空白”（C 端）。
- 新鲜度：商家上架 → 站内可搜，SLA ≤ 5min（商家）。
- 兜底：ES 挂掉，搜索接口**不能 5xx**，必须有降级路径（SRE / 客服）。

这四条决定了搜索系统不能只是 `SELECT * WHERE name LIKE '%kw%'`。

stressTest/REPORT.md: /product/show 同库 PK 查询基线 62,226 RPS / p95 3ms 作为时延锚点

用户搜“苹果手机”，DB 搜不到 iPhone 13



- 用户脑里是中文，商家上架习惯写英文型号。
- LIKE 不知道“苹果 == Apple == iPhone”，子串不匹配 = 0 命中。
- 这不是用户拼错，是搜索系统欠用户一个“分词 + 同义”能力。
- 业务后果：每天数十万次“明明有货却 0 结果”的搜索，全部计入**静默流失**。

`repository/db/dao/product.go:80-98` 是当前 DB 兜底用的 LIKE 实现

LIKE 的第二个问题：跑不动

指标	数值
product 表行数	956K 行
product 表磁盘大小	364 MB
LIKE '%X%' 能走索引?	否（前缀通配）
/product/list 全表 count 实测	24.5 RPS / p95 2.50s

- LIKE '%kw%' 前缀是通配符，B+ 树用不上，必扫全表。
- 96 万行 + 364 MB 数据，单查就吃满一颗 CPU。
- 对照：同库 /product/show 走主键命中 **62,226 RPS / p95 3ms** ——**2,500 倍差距**。
- 业务感知：列表页转圈 2.5s，用户直接退出，转化率掉 15–30%。

stressTest/REPORT.md: product 956K 行 / 364MB; /product/list 24.5 RPS / 2.50s 反例; /product/show 62,226 RPS / 3ms 锚点

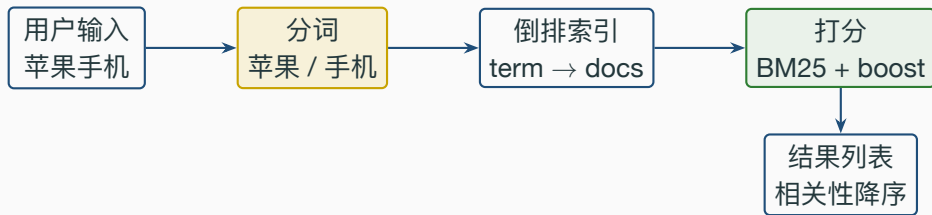
LIKE 缺的三件事，全是业务能力

能力	LIKE 有?	业务后果
分词	无	”苹果手机”搜不到”iPhone”
字段加权	无	标题命中 vs 详情命中等权排
高亮	无	列表里看不到为什么命中

- 分词决定召回，字段加权决定排序，高亮决定点击率——三件事都是**业务旋钮**。
- 三件事任一缺失，”搜索”就退化成”碰运气包含”。
- 这才是 gomall 换 ES 的业务动机，不是图组件新潮。

ES 升级：从词重合到相关性，给商家 与运营留旋钮

ES 替 LIKE 做什么（业务视角）



- 分词：召回不再“碰运气”，“苹果手机”中“手机”二字命中 iPhone 13。
- 倒排索引：单 query 毫秒级返回，p95 落进 200ms 业务 SLA。
- 打分 + 字段加权：运营第一次拿到“调权”旋钮，无需上线即可调首屏。

`repository/es/product_index.go:163-227 SearchProductsWithScore` 多字段加权查询

字段加权背后的业务判断

字段	业务含义	加权
name	商品主名（用户预期匹配）	×3
title	商品标题（含型号 / 规格）	×2
info	商品详情（长文，噪声多）	×1

- 用户搜“iPhone”，“标题就叫 iPhone 13”应当排在“info 提到 iPhone”前面。
- 加权由运营调，不是开发拍。大促可临时把 title 上调到 ×4。
- 不加权 → “标签店家把热词塞进 info”会霸占第一屏 → 用户骂“全是广告”。

`repository/es/product_index.go:174-177 fields={name^3,title^2,info}`

代码段 1: ES 关键词检索 (含 _score 输出)

```
170 must := []map[string]any{{
171     "multi_match": map[string]any{
172         "query": keyword,
173         "fields": []string{"name^3", "title^2", "info"},
174     },
175 }}
176 filter := []map[string]any{}
177 if categoryID != nil {
178     filter = append(filter, map[string]any{
179         "term": map[string]any{"category_id": *categoryID},
180     })
181 }
182 q := map[string]any{
183     "from": from, "size": size,
184     "query": map[string]any{"bool": map[string]any{
185         "must": must, "filter": filter,
186     }},
187 }
```

repository/es/product_index.go:170-191 关键片段

代码段 1 逐行讲解（业务对照）

- **L170–175**: `multi_match` 同时查 `name / title / info`，靠 `^N` 给字段加权——这就是运营“今日特卖 `title` 加权”的代码落点。
- **L176–180**: `filter` 走 `term`，类目过滤**不参与打分**，只做 `yes/no` 过滤。业务上对应“用户在 3C 类目里搜，绝不让美妆混进来”。
- **L182–189**: `bool.must + filter` 是 ES 标准组合——`must` 决定**相关性**，`filter` 决定**可见性**。两者分离，分页 + 打分互不干扰。
- **运营能改 / 不能改**: `fields` 数组里的 `^N` 数字是运营旋钮，`filter` 数组是开发硬约束。这个边界必须写进运营手册。

must / should / filter / must_not 四件套（业务怎么用）

子句	业务用途	gomall 现状
must	关键词召回 / 相关性主链路	已用
should	同义词加分 / 品牌偏好	路线图
filter	类目 / 上架 / 库存 >0 / 价格区间	类目已用
must_not	下架 / 违禁 / 自家店 / 黑名单店	路线图（商家入驻后）

- 业务上的”必须 / 优先 / 限定 / 排除”四件事，正好对应 ES 四个子句。
- 把所有”硬条件”塞进 filter 是最常见的性能优化，可使 QPS 翻倍。
- must_not 配上”自家店”是商家自助上架后的**合规要求**（防自买自卖刷单）。

repository/es/product_index.go:170-191 当前实现 *must + filter*; *should / must_not* 路线图

BM25 + function_score: 把相关性与业务分挂在一起

- **BM25 (ES 7.x 默认)**: tf 走饱和函数, 词频再高不无脑加分; 长度归一让短 name 不被长 info 压制。运营报” 排序偏长描述” 先查 k1/b 是否被改。
- **字段加权 name^3**: 当前生效旋钮, 大促可临时升 title 到 ^4。
- **function_score**: 相关性 $\times$ 销量 / 评分 / 时效, 挂在 BM25 外面, 不污染相关性调试。
- **召回 / 粗排 / 精排三段拆开背 KPI**: 召回看漏召率、粗排看 NDCG@10、精排看 CTR / CVR。gomall 当前到粗排为止。

运营调过的 boost 必须打到 dashboard, 否则后人查” 为什么排序变了” 无从下手。

repository/es/product_index.go:20-41 mapping 沿用集群默认 BM25; service/search/semantic.go:107-111 是 function_score / 精排的统一挂载点

5min 上架可搜 SLA：商家最关心的事

商家上架后第一句话：我的货搜得到吗

- 商家点”上架”按钮 → 立刻去 App 搜 → **搜不到就慌。**
- 5min 是商家的**容忍上限**：拍照 / 调价 / 改文案的反馈节奏。超过 5min，商家就会反复点上架键 / 给客服发消息。
- 同步写 ES 行不行？**不行**：ES 抖动直接 fail 上架，商家完全失去信心。
- 业务承诺：**DB 写成功 = 上架成功**；**ES 索引慢一点没关系，5min 内追上。**

service/product.go:155, 253, 289 三处商品写入后调 emitProductChanged

5min SLA 的时间预算拆解

环节	时间预算	商家体验
1) 商家点上架 → DB commit	5–20 ms	立返” 上架成功”
2) DB outbox 行 → Publisher 扫到	≤ 1 s	无感
3) Publisher → RMQ 投递	10–500 ms	无感
4) Indexer 消费 → ES UpsertProduct	20–50 ms	无感
5) ES refresh_interval 可见窗口	≤ 1 s	搜索可见
合计 (p95)	~2 s	顺畅
合计 (p99, RMQ 堆积)	~30 s	仍达标
业务 SLA 红线	300 s	必须告警

service/product.go:257–265 emitProductChanged; service/outbox/publisher.go:21–32 PollInterval=1s 默认;

service/search/indexer.go:25 Qos=32

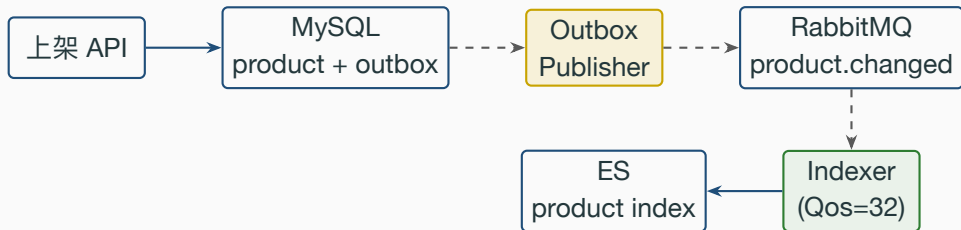
每步如果挂掉，商家看到什么 / 客服怎么答

挂的环节	商家体感	客服应答模板
DB commit	上架按钮转圈 / 报错	” 系统繁忙，请稍后重试”
outbox 不扫	上架成功但永远搜不到	” 后台同步中，5min 内可搜”
RMQ 不通	同上	同上 + 排查 RMQ 监控
Indexer 慢	上架后 1-10min 才搜得到	” 已收到，5min 内同步完成”
ES 全挂	搜得到，但走的 DB LIKE 慢路径	” 搜索系统抢修中，结果仍可用”

- 客服话术分级，对应工程上的故障定位树。
- 任何一步挂，上架本身都不挂——这是 outbox 解耦的业务价值。

`service/product.go:257-265 outbox` 写入；`service/search/indexer.go:32-46` 消费 + Nack 重投

outbox → MQ → indexer 的链路



- 上架事务里只写 product + outbox，二者同库同事务，原子。
- Publisher 后台扫 outbox → 投递到 product.changed 路由。
- Indexer 消费后写 ES。Qos=32 控制并发，避免 ES 被写穿。

service/product.go:257-265 emitProductChanged; service/search/indexer.go:14-49 indexer 主循环

代码段 2: indexer 主循环

```
17 func StartProductIndexer(ctx context.Context) error {
18     rabbitmq.BindDomainQueue(indexerQueue, "product.changed")
19     ch, _ := rabbitmq.GlobalRabbitMQ.Channel()
20     ch.Qos(32, 0, false) // 在途上限 32 条
21     msgs, _ := ch.Consume(indexerQueue, "", false, false, false, false, nil)
22     go func() {
23         for d := range msgs {
24             var ev events.ProductChanged
25             if json.Unmarshal(d.Body, &ev) != nil {
26                 d.Nack(false, false); continue // 脏消息进死信
27             }
28             if handleProductChanged(ctx, ev) != nil {
29                 d.Nack(false, true); continue // 重投自愈
30             }
31             d.Ack(false)
32         }
33     }()
34     return nil
35 }
```

service/search/indexer.go:17-49 (折行省略错误处理)

代码段 2 逐行讲解（业务对照）

- **L18–19**: `BindDomainQueue` 把 `product.changed` 路由绑到 `search.product.indexer` 队列。上架 / 改价 / 下架**共用一条路由**，业务上一致。
- **L22**: `Qos(32, 0, false)` = 给 ES 写入设的**流量阀**。大商家批量上架 1 万个 SKU 时不会冲垮 ES。
- **L30–34**: 脏消息 → `Nack(false, false)` 进死信，**不阻塞健康消息**。运营每天扫一次死信队列。
- **L35–38**: 业务处理失败 → `Nack(false, true)` 重投，对应“ES 短抖动 1s 自愈”。
- **L39**: 成功才 `ack`。索引是**幂等**的（同 `docID` 覆盖），重复消费无害——这是 `at-least-once` 能成立的前提。

service/search/indexer.go:25 Qos=32; :37/:42 Nack 语义; repository/es/product_index.go:115–120

DocumentID = product.ID 决定幂等

代码段 3: product mapping (建索引时的 schema)

```
20 "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
21 "mappings": { "properties": {
22   "name": { "type": "text", "analyzer": "standard" },
23   "title": { "type": "text", "analyzer": "standard" },
24   "info": { "type": "text", "analyzer": "standard" },
25   "category_id": { "type": "long" },
26   "price": { "type": "keyword" },
27   "on_sale": { "type": "boolean" },
28   "created_at": { "type": "date", "format": "epoch_second" }
29 }}
```

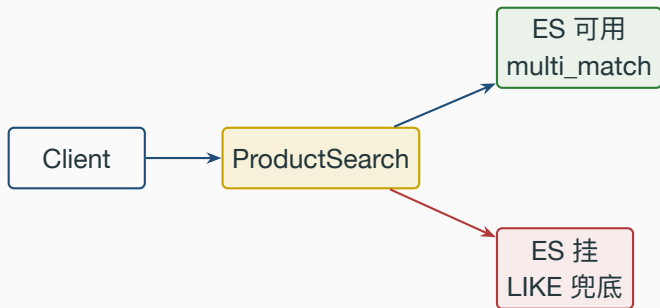
repository/es/product_index.go:20-41 productIndexMapping (节选)

代码段 3 逐行讲解（业务对应）

- **shards=1 / replicas=0**: gomall 单节点部署。生产换多节点要把 replicas 拉到 1。
- **name/title/info 是 text**: 参与分词 + 倒排——商家上架写的文案直接决定能不能被搜到。
- **price 是 keyword**: 不分词，整串做 filter / exact match。价格不会被“分词”成“99” + “元”。
- **category_id / boss_id 是 long**: 按类目 / 按店铺筛，硬过滤。
- **on_sale 是 boolean**: 下架商品要从结果里隐藏——客诉里“为什么搜到下架商品”基本都是这里没维护好。
- **analyzer=standard**: 未装 IK 分词器时的 fallback，保证集群裸装也能跑通。

ES 挂了不能 5xx：降级与运营 SOP

业务约定：搜索接口不许 5xx，启动也不许阻塞



- 5xx 等于 **app 崩页**，比“结果不准”业务损失大得多。LIKE 慢也能用。
- tryInitES 用 defer recover() 把启动 panic 压平成 warn ——ES 故障不阻塞启动。
- **ES 是优化项，不是依赖项。**

ES 全挂的运营 SOP (5 步)

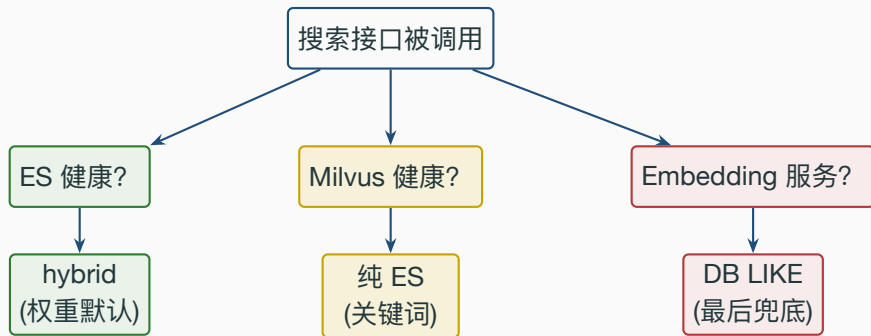
1. **自动降级**: `es.EsClient == nil` → `ProductSearch` 走 `dao.SearchProduct`, 对外仍然 200。
2. **监警告警**: `search_es_fallback_total counter` 跳变 → 触发 P1 告警。
3. **用户体感公告**: 超过 30min 仍未恢复 → App 顶部 banner ” 搜索结果可能不全, 工程师正在抢修”。
4. **客服话术统一**: 内部 IM 推送应答模板, 避免每个客服自行发挥。
5. **恢复后回填**: ES 起来 → 调 `admin/search/backfill` 把降级期间的上架 / 改价回灌索引。

降级期间用户体验差: 搜索很慢 + 中文同义搜不到, 但接口仍可用, 比”白屏 500”业务好 10 倍。

service/product.go:294-325 ES nil → DB LIKE; service/search/backfill.go:13-43 全量回填;

routes/router.go:163 admin/search/backfill

三档降级决策树



ES + Milvus 都

健康 → hybrid；Milvus 挂 → 纯关键词；ES 挂 → DB LIKE 兜底。

对应客服话术：A 档”正常”；B 档”结果可能略有偏差”；C 档”搜索结果可能不全”。

service/search/semantic.go:84-87 ES 失败时 keywordHits=nil； service/product.go:294-323 ES 全挂回 LIKE

**Hybrid 检索：把”商家不会写 SEO”
的锅自己背了**

纯 ES vs 纯向量：业务漏斗对照

策略	优点	业务漏斗损失
纯 LIKE	0 依赖	召回 40–50%，损 GMV 50%+
纯 ES	快 / 加权 / 字段灵活	中文同义召回 60–75%，长尾差
纯向量	语义召回好	短 query 不准、品牌型号搜偏
Hybrid	各取所长	召回 90%+，覆盖短 + 长 query

- 业务上的”搜不到”是召回率问题，”搜偏”是排序问题。
- 纯关键字 / 纯向量都解决半个；hybrid 两个一起解。
- gomall 默认 50:50，给短 query (品牌型号) 和长 query (自然语言) 都留余量。

`service/search/semantic.go:17–23` 默认权重 `weightSemantic=0.5 / weightKeyword=0.5`

ES 关键词搞不定的三类 query

query 类型	例子	ES 行为
自然语言	送女朋友的礼物	拆成”送/女朋友/礼物” → 命中乱
跨表达同义	苹果手机最新款	名义命中 iPhone 15 漏
口语描述	适合健身的运动鞋	长尾词命中率趋近 0

- ES 衡量词重合度，不是语义接近度。
- 用户被短视频 / LLM 养成自然语言搜的习惯，长尾流量比例上升。
- 长尾召回率掉到 60% 以下，整段 GMV 静默漏走，没有告警。

service/search/semantic.go:1-56 hybrid 入口注释与默认权重

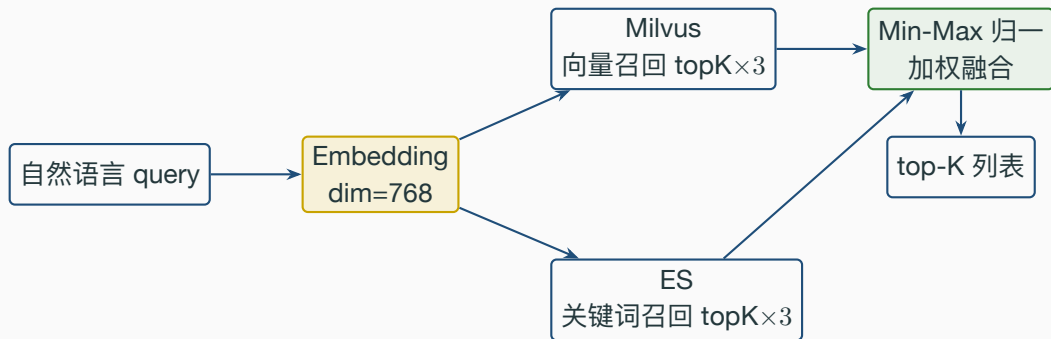
Hybrid 的”商家友好”价值

- **以前**：商家必须写 SEO 优化标题”iPhone 13 苹果手机 5G 全网通国行”才能被搜到。
- **现在**：商家只写”iPhone 13 Pro 256G”，hybrid 用向量召回也能匹配”苹果手机最新款”。
- **商家体验**：少了 30% 文案运营成本，专心做商品本身。
- **用户体验**：自然语言搜得到，不再需要”切换搜索关键词”的耐心。
- 这是 hybrid **对两端都赚**的业务价值，不是技术炫技。

把”商家不会写 SEO”这个能力缺口，由搜索系统自己用 embedding 补上——这是 AI 检索对业务的核心贡献。

service/search/semantic.go:54-143 hybrid 主流程；70-87 embedding + 向量 + 关键词三路召回

hybrid 三层：关键词 + 向量 + 业务调权



向量管召回，关键词管精度，融合权重管业务偏好。service/search/semantic.go:54-143

`semanticSearchWith` 主流程

融合权重：业务调权的入口

权重 (sem : kw)	行为	适合的 query
80 : 20	偏语义	”送女朋友的礼物” 自然语言
50 : 50	中庸	gomall 默认
20 : 80	偏关键词	”iPhone 13 256G” 品牌型号
0 : 100	退化为 ES	ES 全场景兜底

- 50/50 是无偏起点，留 A/B 空间；短 query 偏关键词，长 query 偏语义。
- 大促” 今日特卖” 强插不改算法：在融合分外面加业务分
$$\text{Score} = w_{\text{Sem}} * \text{sem} + w_{\text{Kw}} * \text{kw} + \text{promote}(\text{id})$$
- 不污染长期相关性，运营随时下线。

service/search/semantic.go:21-23 权重常量；:107-111 Score 处可挂业务 boost

代码段 4: min-max 归一 + 加权融合

```
89 semNorm := minMaxNormalize(vecScores(vecHits))
90 kwNorm := minMaxNormalize(esScores(keywordHits))
91 fused := make(map[uint]*types.ProductSemanticHit)
92 for i, h := range vecHits {
93     id := uint(h.ID); if id == 0 { continue }
94     fused[id].SemanticScore = semNorm[i]
95 }
96 for i, h := range keywordHits {
97     fused[h.Doc.ID].KeywordScore = kwNorm[i]
98 }
99 for id, h := range fused {
100     h.Score = weightSemantic*h.SemanticScore +
101             weightKeyword *h.KeywordScore
102 }
```

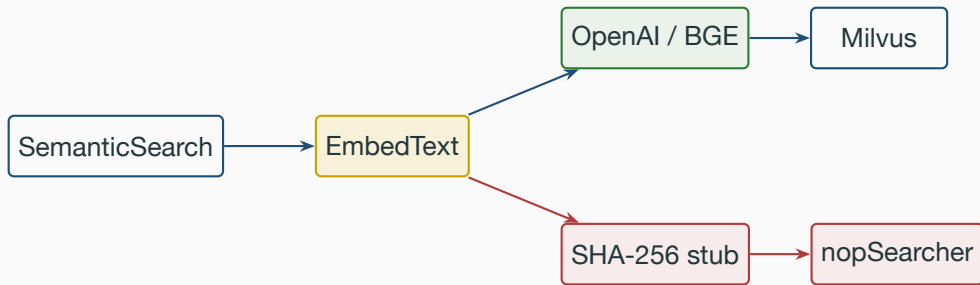
service/search/semantic.go:89-111 融合主体（折行示意）

代码段 4 逐行讲解（业务对照）

- **L89–90**: 两路分数各自 min-max 归一到 $[0, 1]$, 避免 BM25 的 0–100 量纲压死向量的 0–1 量纲——业务上对应”两边权重**真正平等**才能调”。
- **L93–99**: 两路按 product_id 取并集——**仅一路出现的 ID 也计入**（另一路补 0）。业务上对应”向量召回到、ES 没召回到的长尾商品也有机会出现”。
- **L100–103**: 加权融合。weightSemantic / weightKeyword 是常量, **A/B 时换 hash 分桶决定**——运营调权的工程出口。
- **业务对应**: min-max 在小 batch (30–100 条) 下稳定, 单 query 内部即可计算, 无全局统计量——工程上易复制 / 易回滚。

service/search/semantic.go:170–198 minMaxNormalize 全等集合返回 1

Embedding 与 Milvus 都做了可降级



- EMBEDDING_API_URL 未配 → SHA-256 stub 兜底；上线节奏 OpenAI → BGE 省 token；dim=768 硬约束。
- Milvus 走 MilvusSearcher 接口，未注入 → nop 返回 nil → hybrid 退化为纯 ES。
- **Milvus** 挂只丢语义，关键词召回不受影响。

`service/search/embedding.go:52-113`; `service/search/milvus_stub.go:9-47`; `cmd/main.go:86-97` `tryInitMilvus`

HNSW 的三个核心参数（业务旋钮）

参数	含义	gomall 当前值
M	节点最大邻居数	16
efConstruction	建图时候选池大小	200
efSearch	查询时候选池大小	64

- M 大 → 图更密集，召回更准，但内存涨。常见 16–48。
- efConstruction 大 → 建图更慢但更优，回填速度敏感场景调低。
- efSearch 大 → 召回更准延迟更高，是**查询时**唯一可调旋钮——SRE 故障时的**减压阀**。

repository/milvus/product_vector.go:25–28 hnsWM=16 / hnsWEfConstruction=200 / hnsWEfSearch=64

代码段 5: HNSW 索引|建立 + 查询参数

```
80 // 建索引: M=16, efConstruction=200
81 idx, _ := entity.NewIndexHNSW(entity.L2, hnsWM, hnsWEfConstruction)
82 existing, _ := MilvusClient.DescribeIndex(ctx, ProductVectorCollection,
83     productVectorVectorField)
84 if len(existing) == 0 {
85     MilvusClient.CreateIndex(ctx, ProductVectorCollection,
86         productVectorVectorField, idx, false)
87 }
88 MilvusClient.LoadCollection(ctx, ProductVectorCollection, false)
89
90 // 查询: efSearch=64
91 sp, _ := entity.NewIndexHNSWSearchParam(hnsWEfSearch)
92 results, _ := MilvusClient.Search(ctx, ProductVectorCollection,
93     []string{}, expr, []string{"id"},
94     []entity.Vector{entity.FloatVector(queryVec)},
95     "vector", entity.L2, topK, sp)
```

repository/milvus/product_vector.go:80-96 EnsureProductVectorCollection; :141-162 SearchProductVector

代码段 5 逐行讲解（业务对照）

- **L81**: `NewIndexHNSW(L2, M, efConstruction)` 决定索引结构；L2 是欧氏距离，与 BGE / text-embedding-3 默认归一化向量兼容。
- **L82–87**: 先 `DescribeIndex` 判断有没有索引，**没有才建**。
`EnsureProductVectorCollection` 幂等，重启不会重建——SRE 安心运维。
- **L88**: `LoadCollection` 让 collection 进入内存。HNSW 索引 **in-memory**，没 load 就不能查。
- **L91–96**: `NewIndexHNSWSearchParam(efSearch)` 是**查询时调旋钮**——不用重建索引就能调召回精度/延迟权衡。
- **业务对应**: `efSearch` 是 SRE 故障时的**减压阀**，告警来了先调小，召回准确性损 1%–3% 换 P95 减半。

`repository/milvus/product_vector.go:25–28` 三个参数常量

客服话术：三个真实客诉怎么答

客诉案例 1：用户搜不到 + 客服话术

用户原话：“我搜‘蓝牙耳机’一个都没有，你们是不是没货？”

- **后台排查 (30s)**：进 admin 后台搜同词 → 看是否走 ES / 走 LIKE。
- **若 ES 全挂**：业务码无 (HTTP 200)，监控 `search_es_fallback_total`。
- **客服回复模板**：“您看到的是降级结果，搜索系统正在抢修，30 分钟内恢复。您可以按类目浏览。”
- **若 ES 正常但 0 命中**：检查类目过滤是否锁死、检查“蓝牙”是否被分词器分错。
- **客服回复模板**：“您可以试试‘无线耳机’或具体品牌名，系统会推荐更相关的商品。”

service/product.go:294-323 ES nil 走 dao.SearchProduct LIKE 兜底；service/search/semantic.go:84-87 ES 失败 keywordHits=nil

客诉案例 2：商家“我上架了为什么搜不到” + 客服话术

商家原话：“我刚上架的 SKU 12345，搜不到，是不是上架失败？”

- **后台排查 (1min)**: 查 product 表 SKU 12345 存在 → 上架未失败。
- **排查 outbox**: 查 outbox 表对应 product_id 的 product.changed 行 status:
 - pending 5min 内 → 正在同步，等就好
 - pending >5min → Publisher 卡住 / RMQ 不通
 - sent 但 ES 无 → Indexer 报错或 ES 拒收
- **客服回复模板**: “上架已成功，搜索索引同步中，预计 5 分钟内可搜。如超时请回复我们。”
- **兜底操作**: 调 admin/search/backfill 直接灌一次 ES。

*service/product.go:257-265 emitProductChanged 写 outbox; repository/db/dao/outbox.go status 字段
pending/sent/dead; routes/router.go:163 admin/search/backfill*

客诉案例 3：用户搜到下架商品 + 客服话术

用户原话：“我搜到一个商品点进去显示已下架，是不是骗人？”

- **根因**：商家下架 → DB on_sale=false → **outbox 事件未投递 / Indexer 漏消费** → ES 里 on_sale=true 仍在召回。
- **后台排查 (1min)**：DB on_sale 与 ES doc 字段对比，不一致即定位。
- **业务码**：无 (HTTP 200，命中后端再过滤)。
- **客服回复模板**：“该商品已下架，搜索结果尚在更新，5 分钟内会消失。给您带来困扰，赠送 5 元无门槛券。”
- **兜底操作**：手动 DeleteProduct(ID) 删除 ES 文档；事后补 backfill。
- **长期方案**：搜索结果在返回 client 前**再过一遍 DB 校验 on_sale** ——牺牲 5ms 延迟换 0 客诉。

service/search/indexer.go:51-60 handleProductChanged 处理 delete; repository/es/product_index.go:134-150

DeleteProduct

业务边界、容量与个性化路线图

业务边界：本搜索不做什么（诚实清单）

- 无同义词字典：分词靠 standard analyzer，未装 IK 与同义词包——“苹果手机”靠向量召回兜，不靠词典。
- 无搜索历史：用户搜过什么不存，无“最近搜索”功能。
- 无 trending 词：首页搜索框无热词推荐，无“大家都在搜”。
- 无搜索建议 (suggest)：输入框不联想，无前缀补全。
- 无商家 SEO 自助调权重：商家不能自己买“高权重位置”，所有商品按 BM25 + name³ 公平排。
- 无个性化排序：用户画像 / 历史点击 / 协同信号挂载点已留，特征工程链路未接。
- 无以图搜图：CLIP / SigLIP 路线图阶段。

这些“不做”列出来不是甩锅，是划清当前能力边界——后续每接一条都要走 PRD + A/B。

个性化排序路线图：三类特征 + 挂载点

特征类	例子	来源
用户画像	性别 / 年龄段 / 价格带偏好	user profile 表
历史点击	7 天内点过的类目 / 品牌	点击日志 + 实时流
协同信号	同类用户买过什么	离线 i2i / u2i 矩阵

- 历史点击走”近因衰减” $\exp(-(now-ts)/\tau)$, τ 取 3-7 天。
- **挂载点**: `h.Score += wPersonal * personalScore(uid, h.Product)`, 加在融合分外面。
- Redis miss 走默认 0 (保中性), 永远比”用户画像污染基线”安全。

service/search/semantic.go:107-111 融合分赋值处是个性化加权的唯一注入点

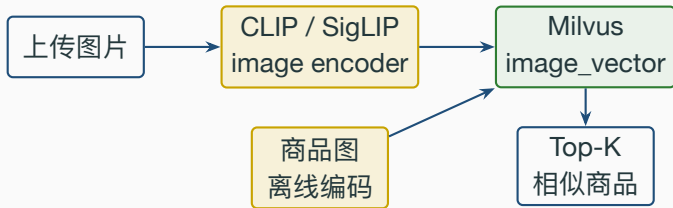
容量规划：1M / 10M / 100M SKU

量级	ES 节点	Milvus 内存	索引重建时长
1M	1 node (4C8G)	4 GB (HNSW)	30 min
10M	3 nodes (8C16G)	32 GB (HNSW)	4 h
100M	5+ nodes (16C32G)	256 GB / 切 IVF_PQ	24 h+

- gomall 当前 **956K** SKU，落在 1M 档，单节点足够。
- 10M 是分水岭：ES 必须分片 + 副本；HNSW 内存接近 32 GB 拐点。
- 100M 切 IVF_PQ 量化索引，召回率换内存；同时考虑冷热分层。

stressTest/REPORT.md: product 956K 行 / 364MB 是 1M 档实测数据

以图搜图路线图 (CLIP / SigLIP)



- CLIP / SigLIP 把图片和文本映射到同一向量空间，可做”图找图”+”文找图”。
- 落地：另建 image_vector collection (dim=512/768)，与文本 collection 解耦。
- 商品图离线批量编码，写入走相同 outbox 路由，复用 indexer 模式。

规划落地点：repository/milvus/ 复用 product_vector 模板新增 image_vector

业务侧总结：四角色 KPI 全打通

一张表把这套搜索系统讲完

能力	实现	代码位置
关键词召回	ES multi_match	repository/es/product_index.go
增量索引	outbox → MQ → indexer	service/search/indexer.go
DB 兜底	ES 客户端为 nil → LIKE	service/product.go:294
全量回填	admin 触发的 Backfill	service/search/backfill.go
语义召回	Milvus + embedding	service/search/semantic.go
模型切换	env: EMBEDDING_API_URL	service/search/embedding.go
启动静默	defer recover	cmd/main.go:64-73

routes/router.go:41-42 search + semantic-search 两个入口; :163 admin/search/backfill

四方视角 + 压测数据复核

- **C 端**: hybrid 召回 90%+, 自然语言搜得到, p95 < 200ms。
- **商家**: 上架 5min 内可搜, outbox 解耦 = 上架本身永远不挂。
- **运营**: name^3 字段加权 / 融合权重 / function_score 三档旋钮齐备。
- **客服 / SRE**: ES / Milvus / Embedding 三档降级, 主流程永不 5xx。
- **同库 PK 查询**能跑 **62,226 RPS / p95 3ms** (product/show), 证明慢的是**查询类型** (LIKE / count), 不是数据库本身。
- **反例** /product/list 全表 count 实测 **24.5 RPS / p95 2.50s**, 验证”搬到 ES”是必选项。

副本系统判据: 副本旧一点点不要命, 副本错乱才要命。outbox 把”上架成功”和”可搜”解耦, 是 5min SLA 的工程根据。

面试 Q&A (1/3)

Q1: 为什么不让上架 API 直接写 ES，而要 outbox？

A: DB 与 ES 不同事务，同步写要么失败回滚（伤体验），要么不一致（伤召回）。outbox 把“DB 写成功”和“ES 写成功”两件事拆开，分别保证。代码：[service/product.go:257-265](#)

Q2: ES 挂掉，搜索接口为什么不直接 5xx？

A: 搜索是入口，5xx 等于 app 崩页。LIKE 慢一点也比白屏强——业务承诺“搜索框永远可用”。代码：[service/product.go:294-323](#)

Q3: hybrid 50/50 这个权重你怎么调？

A: 业务值，不是技术值。先 50/50 跑一轮，A/B 观察 CTR / 转化。短 query 偏关键词，长 query 偏语义。代码：[service/search/semantic.go:21-23](#)

Q4: 商家说“上架后 10 分钟还搜不到”，怎么定位？

A: 四步走：查 product 表确认入库 → 查 outbox 行 status → 查 RMQ 队列深度 → 查 ES doc。每步对应一个客服话术（见客诉案例 2）。代码：[repository/db/dao/outbox.go](#) status 字段

面试 Q&A (2/3)

Q5: min-max 归一在什么场景会失效?

A: 所有分数等值时 span=0。代码里返回常量 1，意味着“两路一样可信”，让另一路决定排序。代码: `service/search/semantic.go:186-192`

Q6: HNSW 的 efSearch 你怎么调?

A: efSearch 是查询时旋钮，不用重建索引。延迟告警时调小 (64→32) 召回率损 1%-3%，P95 减半。代码: `repository/milvus/product_vector.go:141`

Q7: Milvus 挂了怎么办? 为什么不直接 5xx?

A: nopMilvusSearcher 返回空，hybrid 退化为纯 ES。关键词召回不受影响，只丢自然语言长尾。代码: `service/search/milvus_stub.go:19-24`

Q8: 搜索结果命中下架商品 (客诉案例 3) 根因有几种?

A: 三种: DB 改 on_sale 但 outbox 漏写、outbox 写了但 indexer 没消费、indexer 调 DeleteProduct 但 ES 拒收。长期方案: 返回前再过一次 DB 校验。代码: `service/search/indexer.go:51-60`

面试 Q&A (3/3)

Q9: 5 分钟新鲜度 SLA 在哪一步可能掉?

A: 堆积一般出现在 RMQ 与 indexer 之间, Qos=32 + ES refresh_interval=1s 下 p95 < 30s。监控 `search.product.indexer.queue_depth`。代码: `service/search/indexer.go:25`

Q10: 100M SKU 时这个架构还能用吗?

A: ES 切多分片 + 副本; HNSW 内存撑不住改 IVF_PQ 量化; 冷热分层把老 SKU 放冷索引。`repository/milvus/product_vector.go:80 NewIndexHNSW`

Q11: 为什么不接同义词字典 / 搜索历史 / trending?

A: 业务边界。当前流量没到拐点, 同义词靠 hybrid 向量召回兜, 搜索历史涉及用户隐私单独立项, trending 词运营手动维护轮播。详见”业务边界” 帧。`service/search/semantic.go`
`hybrid` 主入口

Q12: mapping 改了字段类型怎么办?

A: ES mapping 字段类型不能原地改, 必须 `reindex` 或新建索引 + `alias` 切换。回填走 `admin/search/backfill`。`routes/router.go:163`

代码位置总览 + 配套材料

搜索 / 语义入口 *routes/router.go:41–42*

ES 检索 + mapping *repository/es/product_index.go:20–41, 163–227*

outbox 事件 *service/product.go:257–265*

indexer 消费 *service/search/indexer.go:17–49*

ES 兜底分支 *service/product.go:294–323*

全量回填 *service/search/backfill.go:13–43*

hybrid 融合 + embedding *service/search/semantic.go, embedding.go*

Milvus HNSW *repository/milvus/product_vector.go:25–28, 80–96, 141–162*

Milvus 接口抽象 *service/search/milvus_stub.go:9–47*

启动静默 *cmd/main.go:64–73 + initialize/search.go:13–31*

业务码表 *pkg/e/code.go:46–58*

配套： deck 04 outbox / saga； deck 10 流量治理与降级； deck 11 一致性；
stressTest/REPORT.md； docs/blog/10-milvus-vector-search.md；
docs/blog/11-semantic-search.md。